

第三阶段报告：数据库逻辑结构定稿

项目名称：复旦校园百事通问答系统

小组成员：洪东浩、马天行

提交日期：2026年4月26日

GitHub仓库：https://github.com/hdh-fall/Fudan-Campus-Info-System

一、完整表结构说明

基于前两阶段的需求分析与关系模式设计，我们对10张核心表进行了最终逻辑定稿。本章详述每张表的字段、类型、约束及其背后的业务规则，确保数据库层能够严格、完整地表达系统需求。所有建表语句均基于 **MySQL 8.0** 语法编写。

1. department (院系表)

设计目的： 消除用户、教师、课程中院系名的冗余，实现院系数据的集中管理。

字段名	类型	约束	说明
department_id	INT	PRIMARY KEY , AUTO_INCREMENT	院系唯一标识
name	VARCHAR(100)	NOT NULL, UNIQUE	院系全称，如“计算机科学技术学院”

业务规则： 院系名称在全校范围内唯一，不允许为空。

2. user (用户表)

设计目的： 存储普通用户与管理员账户，支持身份鉴权与个人信息维护。

字段名	类型	约束	说明
user_id	INT	PRIMARY KEY , AUTO_INCREMENT	用户唯一标识
username	VARCHAR(50)	NOT NULL, UNIQUE	登录名
name	VARCHAR(50)	NOT NULL	真实姓名
grade	VARCHAR(20)	CHECK (grade REGEXP '^[0-9]{4}级\$')	年级，如“2024级”
department_id	INT	FOREIGN KEY → department(department_id) ON DELETE RESTRICT	所属院系
role	ENUM('user','admin')	NOT NULL, DEFAULT 'user'	角色：普通用户/管理员
email	VARCHAR(100)	CHECK (email LIKE '%_@%.%')	电子邮箱

字段名	类型	约束	说明
phone	VARCHAR(20)	-	联系电话
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	注册时间

业务规则：

- 登录名全局唯一，用于身份切换时的精准识别。
- 年级字段通过 CHECK 约束确保格式统一（如“2024级”），杜绝随意填写。
- 角色字段使用 ENUM 并设置默认值，保证非管理员用户的默认权限。
- 邮箱通过 CHECK 约束进行简单格式校验，避免无效数据入库。
- 外键采用 RESTRICT，防止误删院系导致用户数据悬空。

3. campus（校区表）

设计目的：记录学校各个校区的基础信息，支持按校区聚合建筑与活动。

字段名	类型	约束	说明
campus_id	INT	PRIMARY KEY, AUTO_INCREMENT	校区唯一标识
name	VARCHAR(50)	NOT NULL, UNIQUE	校区名称，如“邯郸校区”
address	VARCHAR(200)	-	详细地址
contact_phone	VARCHAR(20)	-	联系电话
latitude	DECIMAL(10,8)	CHECK (latitude BETWEEN -90 AND 90)	纬度
longitude	DECIMAL(11,8)	CHECK (longitude BETWEEN -180 AND 180)	经度
description	TEXT	-	校区描述

业务规则：校区名唯一。经纬度通过 CHECK 约束确保合法范围，为未来地图集成提供可信数据。

4. building（建筑表）

设计目的：描述校区内的各类建筑，通过外键与校区构成一对多关系。

字段名	类型	约束	说明
building_id	INT	PRIMARY KEY, AUTO_INCREMENT	建筑唯一标识
name	VARCHAR(100)	NOT NULL	建筑名称，如“光华楼”
type	VARCHAR(50)	-	类型：教学楼、图书馆等
campus_id	INT	NOT NULL, FOREIGN KEY → campus(campus_id) ON DELETE RESTRICT	所属校区

字段名	类型	约束	说明
floors	INT	CHECK (floors >= 0)	楼层数
open_time	TIME	–	开放时间
close_time	TIME	–	关闭时间
description	TEXT	–	建筑描述

业务规则：

- 必须有对应校区，外键采用 RESTRICT，欲删除校区前需先处理该校区下所有建筑。
- 楼层数不允许为负。
- 开放/关闭时间使用 TIME 类型，便于时间区间查询。

5. facility（设施表）

设计目的： 记录建筑内的具体设施（食堂、咖啡厅、自习室等），与建筑构成一对多关系。

字段名	类型	约束	说明
facility_id	INT	PRIMARY KEY , AUTO_INCREMENT	设施唯一标识
name	VARCHAR(100)	NOT NULL	设施名称
type	VARCHAR(50)	NOT NULL	类型：咖啡厅、自习室等
building_id	INT	NOT NULL, FOREIGN KEY → building(building_id) ON DELETE CASCADE	所属建筑
open_time	VARCHAR(100)	–	开放时间文本，如“10:00-20:00”
location_desc	VARCHAR(200)	–	位置描述，如“一楼东侧”
capacity	INT	CHECK (capacity >= 0)	容量（座位数）
contact	VARCHAR(50)	–	联系方式
description	TEXT	–	设施描述

业务规则：

- 设施必须归属于某一建筑；建筑被删除时，其内部设施自然失效，故使用 CASCADE。
- 开放时间采用自由文本以兼容不规则时间描述（如“午餐 11:00-13:00, 晚餐 17:00-19:00”），这是一种有意识的非严格结构化权衡。
- 容量通过 CHECK 约束保证非负。

6. teacher（教师表）

设计目的： 管理教师个人资料，关联院系及职称。

字段名	类型	约束	说明
teacher_id	INT	PRIMARY KEY , AUTO_INCREMENT	教师唯一标识
name	VARCHAR(50)	NOT NULL	教师姓名
department_id	INT	FOREIGN KEY → department(department_id) ON DELETE RESTRICT	所属院系
title	VARCHAR(50)	–	职称：教授、副教授等
email	VARCHAR(100)	CHECK (email LIKE '%_@%.%')	邮箱
phone	VARCHAR(20)	–	电话

业务规则： 外键参照院系，使用 RESTRICT 防止因误删院系而导致教师记录孤立。

7. course（课程表）

设计目的： 存储课程基本信息，通过院系外键实现归属管理。

字段名	类型	约束	说明
course_id	INT	PRIMARY KEY , AUTO_INCREMENT	课程唯一标识
name	VARCHAR(100)	NOT NULL	课程名称
department_id	INT	FOREIGN KEY → department(department_id) ON DELETE RESTRICT	开课院系
credits	DECIMAL(3,1)	CHECK (credits > 0 AND credits <= 20)	学分
description	TEXT	–	课程简介
semester_offered	VARCHAR(20)	–	建议学期，如“2025-2026-1”

业务规则：

- 学分必须为正数，且不得超过20（防止异常输入）。
- 课程与院系关系采用 RESTRICT，保证数据完整性。

8. course_teacher（课程-教师授课关系表，多对多）

设计目的： 实现课程与教师的多对多关联，同时记录授课学期与角色。

字段名	类型	约束	说明
course_id	INT	PRIMARY KEY , FOREIGN KEY → course(course_id) ON DELETE CASCADE	课程ID
teacher_id	INT	PRIMARY KEY , FOREIGN KEY → teacher(teacher_id) ON DELETE CASCADE	教师ID

字段名	类型	约束	说明
semester	VARCHAR(20)	PRIMARY KEY	授课学期，如“2025-2026-1”
role	VARCHAR(30)	DEFAULT '主讲'	角色：主讲、助教等
remarks	TEXT	–	备注

业务规则：

- 复合主键 (course_id, teacher_id, semester) 唯一标识一次授课安排。
- 学期纳入主键，允许同一教师在不同学期重复教授同一课程。
- 外键采用 CASCADE，课程或教师删除时，对应的授课记录自动清除，保持数据一致性。

9. event（活动表）

设计目的： 记录校园各类活动，可灵活关联校区或建筑，支持讲座、论坛等分类。

字段名	类型	约束	说明
event_id	INT	PRIMARY KEY , AUTO_INCREMENT	活动唯一标识
name	VARCHAR(100)	NOT NULL	活动名称
event_time	DATETIME	NOT NULL	举办时间
location_desc	VARCHAR(200)	–	位置描述（自由文本）
campus_id	INT	FOREIGN KEY → campus(campus_id) ON DELETE SET NULL	所属校区（可空）
building_id	INT	FOREIGN KEY → building(building_id) ON DELETE SET NULL	所属建筑（可空）
organizer	VARCHAR(100)	–	主办单位
category	VARCHAR(50)	–	类别：讲座、论坛、社团等
description	TEXT	–	活动描述

业务规则：

- 活动时间不可为空，确保按时间排序与统计的可靠性。
- 活动与校区、建筑关系为可选关联；当父记录被删除时，外键自动置为 NULL（SET NULL），保留活动记录本身。
- 保留 location_desc 自由文本字段，即使外键被置空，仍能提供基本位置信息。

10. query_record（查询记录表）

设计目的： 记录用户每一次查询，为历史记录查看与热门统计提供数据基础。

字段名	类型	约束	说明
record_id	INT	PRIMARY KEY , AUTO_INCREMENT	记录唯一标识
user_id	INT	NOT NULL, FOREIGN KEY → user(user_id) ON DELETE CASCADE	用户ID
question	TEXT	NOT NULL	用户输入的问题或关键词
query_time	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	查询时间
category	VARCHAR(50)	–	系统识别的类别： FACILITY_CANTEEN、 FACILITY_LIBRARY、 FACILITY_STUDY_ROOM、 FACILITY_LAB等10种枚举值
result_summary	VARCHAR(200)	–	返回结果的摘要
used_nl2sql	BOOLEAN	DEFAULT FALSE	是否使用了NL2SQL技术
client_ip	VARCHAR(45)	–	客户端IP，支持IPv6

业务规则：

- 用户删除时，其所有查询记录随之删除（CASCADE），保护隐私并避免孤儿数据。
- **query_time** 默认记录当前时间，确保日志准确性。
- **category** 用于后续热门类别统计，是聚合查询的关键字段。系统支持10种查询类别：
 - FACILITY_CANTEEN（食堂）、FACILITY_CAFE（咖啡厅）、FACILITY_LIBRARY（图书馆）
 - FACILITY_STUDY_ROOM（自习室）、FACILITY_LAB（实验室）
 - BUILDING（建筑）、CAMPUS（校区）
 - COURSE（课程）、TEACHER（教师）、EVENT（活动）
- **used_nl2sql** 为 AI 问答功能的效果评估提供标记。

11. query_statistics_cache（查询统计缓存表）

设计目的： 缓存 **query_record** 的日度聚合统计结果，避免前端每次实时计算，提升趋势图表与排行榜的查询性能。

字段名	类型	约束	说明
stat_id	INT	PRIMARY KEY , AUTO_INCREMENT	记录唯一标识
stat_date	DATE	NOT NULL	统计日期
category	VARCHAR(50)	–	查询类别
query_count	INT	DEFAULT 0	该日期该类别的查询次数
unique_users	INT	DEFAULT 0	该日期该类别的独立用户数

字段名	类型	约束	说明
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	记录创建时间

业务规则：

- 唯一约束 (stat_date, category) 保证同一天同一类别只有一条汇总记录，插入冲突时自动更新计数。
- 本表不直接由应用层写入，而是由触发器 trg_after_query_insert 在每次新增查询记录时实时累加，以及由存储过程 sp_update_query_statistics 定期全量刷新。
- 索引 idx_stat_date 加速按日期范围的趋势查询。

二、主要约束设计说明

1. 主键策略

- 所有实体表均采用 **单列自增整数** 作为代理主键，结构统一，便于索引与外键引用。
- 多对多关系表 course_teacher 使用 **复合主键 (course_id, teacher_id, semester)**，完美刻画业务唯一性，防止同一学期内重复排课。

2. 外键与参照完整性

- RESTRICT** (7处)：用于院系、校区等核心实体的外键，防止误删除导致子表数据孤立，维护数据核心骨架的安全。
- CASCADE** (5处)：用于强依赖关系（建筑→设施、用户→查询记录、课程/教师→授课记录），父记录删除时子记录自动清理，简化运维，符合业务直觉。
- SET NULL** (2处)：用于活动与校区/建筑的可选关联，父记录删除后保留活动记录，同时通过外键置空标识关联失效，兼顾数据灵活性与历史保留需求。

3. 业务规则约束

约束类型	应用示例	业务意义
NOT NULL	user.name, building.campus_id, query_record.question	保证关键信息完整，避免程序空指针异常。
UNIQUE	department.name, campus.name, user.username	消除同名冲突，配合应用层进行唯一性校验。
CHECK	user.grade (正则格式), user.email (基本格式), credits (0-20), capacity (≥0)	在数据库层拦截明显的脏数据，减轻应用层校验压力。
DEFAULT	user.role='user', query_record.used_nl2sql=FALSE	合理设缺省，减少插入时的必填字段，提高开发效率。
ENUM	user.role	限定角色取值范围，防止拼写错误导致业务逻辑紊乱。

4. 索引设计理由

为高频查询字段和外键建立索引，确保多表 JOIN 和聚合统计的效率：

- `building(campus_id)`、`facility(building_id)`：支撑“按校区找建筑”“按建筑找设施”的常见导航查询。
- `teacher(department_id)`、`course(department_id)`：加速“某院系下教师/课程”的统计。
- `event(event_time)`：支持“近期活动”时间范围扫描。
- `query_record(user_id, query_time)`：联合索引同时覆盖“某用户历史记录”和按时间排序。
- `course_teacher(teacher_id)`：加速“某教师所授课程”的查询。
- `query_record(category)`：加速热门查询类别的聚合统计。
- `query_record(query_time)`：加速按时间范围的查询趋势分析。
- `event(category, event_time)`：复合索引支撑活动类别与时间的联合查询，服务于个性化推荐存储过程。
- `query_statistics_cache(stat_date)`：加速按日期范围的统计趋势查询。

所有索引均已整合到最终的 DDL 脚本中。

5. 规范化与有意识权衡

- 所有表均通过 3NF 检查，消除了院系名等关键数据的冗余，避免更新/删除异常。
- `facility.open_time` 采用 VARCHAR 自由文本而非结构化时间类型，是 **为兼容不规律开放时间描述所做的有意识权衡**，虽然牺牲了严格的时间计算能力，但更贴合校园设施的真实情况。
- `event.location_desc` 与外键并存，既保留了结构化查询能力，又保障了外键置空后的可读性，属于典型的 **查询便利性冗余**，不会引发更新异常。

三、核心 SQL 草稿（DDL 与初始索引）

以下 SQL 可作为项目的 `schema.sql` 文件直接执行，完整构建数据库结构。

```
-- =====
-- 复旦校园百事通问答系统 - 数据库模式定义
-- 适用环境：MySQL 8.0+
-- =====

CREATE DATABASE IF NOT EXISTS fudan_campus_info
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;
USE fudan_campus_info;

-- -----
-- 1. 院系表
-- -----

CREATE TABLE department (
  department_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;

-- -----
-- 2. 用户表
-- -----

CREATE TABLE user (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
```

```

username VARCHAR(50) NOT NULL UNIQUE,
name VARCHAR(50) NOT NULL,
grade VARCHAR(20) CHECK (grade REGEXP '^[0-9]{4}级$'),
department_id INT,
role ENUM('user','admin') NOT NULL DEFAULT 'user',
email VARCHAR(100) CHECK (email LIKE '%_@_%._%'),
phone VARCHAR(20),
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (department_id) REFERENCES department(department_id)
    ON DELETE RESTRICT
) ENGINE=InnoDB;

```

-- 3. 校区表

```

CREATE TABLE campus (
    campus_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,
    address VARCHAR(200),
    contact_phone VARCHAR(20),
    latitude DECIMAL(10,8) CHECK (latitude BETWEEN -90 AND 90),
    longitude DECIMAL(11,8) CHECK (longitude BETWEEN -180 AND 180),
    description TEXT
) ENGINE=InnoDB;

```

-- 4. 建筑表

```

CREATE TABLE building (
    building_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    type VARCHAR(50),
    campus_id INT NOT NULL,
    floors INT CHECK (floors >= 0),
    open_time TIME,
    close_time TIME,
    description TEXT,
    FOREIGN KEY (campus_id) REFERENCES campus(campus_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

-- 5. 设施表

```

CREATE TABLE facility (
    facility_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    type VARCHAR(50) NOT NULL,
    building_id INT NOT NULL,
    open_time VARCHAR(100),
    location_desc VARCHAR(200),
    capacity INT CHECK (capacity >= 0),
    contact VARCHAR(50),
    description TEXT,

```

```

        FOREIGN KEY (building_id) REFERENCES building(building_id)
        ON DELETE CASCADE
    ) ENGINE=InnoDB;

-----
-- 6. 教师表
-----

CREATE TABLE teacher (
    teacher_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    department_id INT,
    title VARCHAR(50),
    email VARCHAR(100) CHECK (email LIKE '%_@__%.__%'),
    phone VARCHAR(20),
    FOREIGN KEY (department_id) REFERENCES department(department_id)
    ON DELETE RESTRICT
) ENGINE=InnoDB;

-----
-- 7. 课程表
-----

CREATE TABLE course (
    course_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    department_id INT,
    credits DECIMAL(3,1) CHECK (credits > 0 AND credits <= 20),
    description TEXT,
    semester_offered VARCHAR(20),
    FOREIGN KEY (department_id) REFERENCES department(department_id)
    ON DELETE RESTRICT
) ENGINE=InnoDB;

-----
-- 8. 课程-教师授课关系表（多对多）
-----

CREATE TABLE course_teacher (
    course_id INT,
    teacher_id INT,
    semester VARCHAR(20),
    role VARCHAR(30) DEFAULT '主讲',
    remarks TEXT,
    PRIMARY KEY (course_id, teacher_id, semester),
    FOREIGN KEY (course_id) REFERENCES course(course_id)
    ON DELETE CASCADE,
    FOREIGN KEY (teacher_id) REFERENCES teacher(teacher_id)
    ON DELETE CASCADE
) ENGINE=InnoDB;

-----
-- 9. 活动表
-----

CREATE TABLE event (
    event_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,

```

```

    event_time DATETIME NOT NULL,
    location_desc VARCHAR(200),
    campus_id INT,
    building_id INT,
    organizer VARCHAR(100),
    category VARCHAR(50),
    description TEXT,
    FOREIGN KEY (campus_id) REFERENCES campus(campus_id)
        ON DELETE SET NULL,
    FOREIGN KEY (building_id) REFERENCES building(building_id)
        ON DELETE SET NULL
) ENGINE=InnoDB;

```

```

-- -----
-- 10. 查询记录表
-- -----

```

```

CREATE TABLE query_record (
    record_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    question TEXT NOT NULL,
    query_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    category VARCHAR(50),
    result_summary VARCHAR(200),
    used_nl2sql BOOLEAN DEFAULT FALSE,
    client_ip VARCHAR(45),
    FOREIGN KEY (user_id) REFERENCES user(user_id)
        ON DELETE CASCADE
) ENGINE=InnoDB;

```

```

-- -----
-- 索引（除主键外）
-- -----

```

```

CREATE INDEX idx_building_campus ON building(campus_id);
CREATE INDEX idx_facility_building ON facility(building_id);
CREATE INDEX idx_course_department ON course(department_id);
CREATE INDEX idx_teacher_department ON teacher(department_id);
CREATE INDEX idx_event_time ON event(event_time);
CREATE INDEX idx_query_user_time ON query_record(user_id, query_time);
CREATE INDEX idx_course_teacher_teacher ON course_teacher(teacher_id);
CREATE INDEX idx_query_category ON query_record(category);
CREATE INDEX idx_query_time_only ON query_record(query_time);
CREATE INDEX idx_event_category_time ON event(category, event_time);

```

```

-- -----
-- 11. 查询统计缓存表（分析增强模块）
-- -----

```

```

CREATE TABLE query_statistics_cache (
    stat_id INT AUTO_INCREMENT PRIMARY KEY,
    stat_date DATE NOT NULL,
    category VARCHAR(50),
    query_count INT DEFAULT 0,
    unique_users INT DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uk_date_category (stat_date, category),

```

```
INDEX idx_stat_date (stat_date)
) ENGINE=InnoDB;
```

关键业务查询草稿示例

为验证数据库对项目需求的支撑能力，此处列出几个核心查询的 SQL 草稿：

1. 查询邯郸校区所有建筑及其设施数量

```
SELECT b.name AS building, COUNT(f.facility_id) AS facility_count
FROM building b
LEFT JOIN facility f ON b.building_id = f.building_id
JOIN campus c ON b.campus_id = c.campus_id
WHERE c.name = '邯郸校区'
GROUP BY b.building_id;
```

2. 统计各院系课程数量（聚合查询）

```
SELECT d.name AS department, COUNT(c.course_id) AS course_count
FROM department d
LEFT JOIN course c ON d.department_id = c.department_id
GROUP BY d.department_id;
```

3. 查询某位教师的所有授课课程（多表连接）

```
SELECT t.name AS teacher, c.name AS course, ct.semester, ct.role
FROM teacher t
JOIN course_teacher ct ON t.teacher_id = ct.teacher_id
JOIN course c ON ct.course_id = c.course_id
WHERE t.teacher_id = ?;
```

4. 统计热门查询类别（业务统计）

```
SELECT category, COUNT(*) AS query_count
FROM query_record
WHERE query_time >= DATE_SUB(NOW(), INTERVAL 30 DAY)
GROUP BY category
ORDER BY query_count DESC
LIMIT 10;
```

5. 查找下周即将举办的活动

```
SELECT name, event_time, location_desc, organizer
FROM event
WHERE event_time BETWEEN NOW() AND DATE_ADD(NOW(), INTERVAL 7 DAY)
ORDER BY event_time;
```

以上 SQL 草稿覆盖了基础查询、多表连接、聚合统计及时序过滤等典型场景，验证了当前数据库设计已经能够完整支撑后续系统功能的开发。

数据库高级对象定义（分析增强模块）

为支撑数据分析与可视化功能，系统还定义了以下视图、存储过程、触发器和事件。

视图 (Views)

```
-- 热门查询类别统计
CREATE OR REPLACE VIEW v_popular_query_categories AS
SELECT
    category,
    COUNT(*) AS query_count,
    COUNT(DISTINCT user_id) AS unique_users,
    MAX(query_time) AS last_query_time,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM query_record), 2) AS percentage
FROM query_record
WHERE category IS NOT NULL AND category != ''
GROUP BY category
ORDER BY query_count DESC;

-- 每日查询趋势
CREATE OR REPLACE VIEW v_daily_query_trend AS
SELECT
    DATE(query_time) AS query_date,
    COUNT(*) AS daily_count,
    COUNT(DISTINCT user_id) AS unique_users,
    SUM(CASE WHEN used_nl2sql = TRUE THEN 1 ELSE 0 END) AS nl2sql_count,
    SUM(CASE WHEN used_nl2sql = FALSE THEN 1 ELSE 0 END) AS manual_count
FROM query_record
GROUP BY DATE(query_time)
ORDER BY query_date DESC;

-- 活跃用户排行
CREATE OR REPLACE VIEW v_active_users AS
SELECT
    u.user_id, u.username, u.name,
    COUNT(qr.record_id) AS total_queries,
    COUNT(DISTINCT qr.category) AS categories_explored
FROM user u
LEFT JOIN query_record qr ON u.user_id = qr.user_id
GROUP BY u.user_id, u.username, u.name
HAVING total_queries > 0
ORDER BY total_queries DESC;
```

```
-- 校区设施热度
CREATE OR REPLACE VIEW v_campus_facility_popularity AS
SELECT
    c.campus_id, c.name AS campus_name,
    COUNT(DISTINCT b.building_id) AS building_count,
    COUNT(DISTINCT f.facility_id) AS facility_count
FROM campus c
LEFT JOIN building b ON c.campus_id = b.campus_id
LEFT JOIN facility f ON b.building_id = f.building_id
GROUP BY c.campus_id, c.name
ORDER BY facility_count DESC;
```

```
-- 课程受欢迎程度
CREATE OR REPLACE VIEW v_course_popularity AS
SELECT
    co.course_id, co.name AS course_name,
    COUNT(DISTINCT ct.teacher_id) AS teacher_count,
    COUNT(ct.semester) AS total_semesters
FROM course co
LEFT JOIN course_teacher ct ON co.course_id = ct.course_id
GROUP BY co.course_id, co.name, co.credits
ORDER BY teacher_count DESC, total_semesters DESC;
```

存储过程 (Stored Procedures)

```
-- 更新查询统计缓存（使用游标）
CREATE PROCEDURE sp_update_query_statistics(IN p_date DATE)
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_category VARCHAR(50);
    DECLARE v_count INT;
    DECLARE v_users INT;
    DECLARE cur_stats CURSOR FOR
        SELECT category, COUNT(*), COUNT(DISTINCT user_id)
        FROM query_record
        WHERE DATE(query_time) = p_date
        AND category IS NOT NULL AND category != ''
        GROUP BY category;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
    OPEN cur_stats;
    read_loop: LOOP
        FETCH cur_stats INTO v_category, v_count, v_users;
        IF done THEN LEAVE read_loop; END IF;
        INSERT INTO query_statistics_cache (stat_date, category, query_count,
unique_users)
            VALUES (p_date, v_category, v_count, v_users)
            ON DUPLICATE KEY UPDATE query_count = v_count, unique_users = v_users;
        END LOOP;
    CLOSE cur_stats;
END;
```

```

-- 个性化活动推荐（使用临时表）
CREATE PROCEDURE sp_get_personalized_recommendations(
    IN p_user_id INT, IN p_limit INT
)
BEGIN
    CREATE TEMPORARY TABLE IF NOT EXISTS temp_user_preferences AS
    SELECT category, COUNT(*) AS pref_count
    FROM query_record
    WHERE user_id = p_user_id AND category IS NOT NULL
    GROUP BY category ORDER BY pref_count DESC LIMIT 3;
    SELECT e.event_id, e.name, e.event_time, e.category
    FROM event e
    WHERE e.category IN (SELECT category FROM temp_user_preferences)
    AND e.event_time >= NOW()
    ORDER BY e.event_time ASC LIMIT p_limit;
    DROP TEMPORARY TABLE IF EXISTS temp_user_preferences;
END;

```

触发器 (Triggers)

```

-- 查询记录插入后自动更新统计缓存
CREATE TRIGGER trg_after_query_insert
AFTER INSERT ON query_record
FOR EACH ROW
BEGIN
    INSERT INTO query_statistics_cache (stat_date, category, query_count,
unique_users)
    VALUES (DATE(NEW.query_time), NEW.category, 1, 1)
    ON DUPLICATE KEY UPDATE
        query_count = query_count + 1,
        unique_users = (
            SELECT COUNT(DISTINCT user_id) FROM query_record
            WHERE DATE(query_time) = DATE(NEW.query_time)
            AND category = NEW.category
        );
END;

```

事件调度器 (Events)

```

-- 每日凌晨自动更新统计（需 SET GLOBAL event_scheduler = ON;）
CREATE EVENT IF NOT EXISTS evt_daily_statistics_update
ON SCHEDULE EVERY 1 DAY
STARTS TIMESTAMP(CURRENT_DATE + INTERVAL 1 DAY, '00:00:00')
DO
BEGIN
    CALL sp_update_query_statistics(DATE(NOW() - INTERVAL 1 DAY));
END;

```

四、后续开发支撑说明

- **数据结构完整性**: 10 张核心表及 1 张统计缓存表均已全面实现，可直接作为后端 ORM 模型或手写 SQL 的基础。
- **数据测试准备**: `schema.sql` 可以一键部署，`analytics_enhancement.sql` 可追加分析增强功能；配合 CSV 批量导入脚本（管理员功能）可快速生成 50+ 条测试数据。
- **功能联动**: 所有表间关系均通过外键约束保障，前后端在增删改操作时无需担心数据孤立问题，业务逻辑可专注于应用层验证。
- **性能预估**: 针对高频查询的外键和分类字段已建立索引，在千级数据量下响应速度有充分保障，后续可根据实际负载通过 `EXPLAIN` 调优。
- **扩展预留**: `query_record` 中预留的 `used_nl2sql` 和 `category` 字段，为后续 AI 问答功能的评估与优化提供了数据埋点。
- **分析增强模块**: 视图、存储过程、触发器和事件调度器均为数据库层的封装，前端通过 REST API 调用即可获取聚合结果，应用层无需重复计算，架构清晰且易于扩展。

五、扩展功能说明

5.1 AI智能问答

系统集成通义千问大模型，实现了自然语言到SQL的转换（NL2SQL）：

技术架构

- **前端**: Vue 3 + Element Plus，提供友好的自然语言输入界面
- **后端**: Spring Boot + 通义千问API，负责语义理解和SQL生成
- **数据库**: MySQL 8.0，存储查询历史和执行结果

工作流程

1. 用户输入自然语言问题（如“复旦有哪些食堂？”）
2. 后端调用通义千问API，将自然语言转换为SQL查询
3. 执行SQL查询并返回结果
4. 保存查询记录到 `query_record` 表，标记 `used_nl2sql = TRUE`
5. 如果AI服务不可用，自动降级到规则匹配模式

类别识别

系统支持10种查询类别的自动识别：

- **设施类**: FACILITY_CANTEEN（食堂）、FACILITY_CAFE（咖啡厅）、FACILITY_LIBRARY（图书馆）、FACILITY_STUDY_ROOM（自习室）、FACILITY_LAB（实验室）
- **建筑类**: BUILDING（建筑）、CAMPUS（校区）
- **教学类**: COURSE（课程）、TEACHER（教师）
- **活动类**: EVENT（活动）

中文映射

- 数据库字段名自动转换为中文显示（如 `course_name` → “课程名称”）
- 时间字段格式化为友好格式（如 `2026-05-21 14:30:00` → “2026年5月21日 14:30”）

5.2 个性化推荐系统

基于用户查询历史，系统实现了智能推荐算法：

推荐算法

1. **数据收集**：从 `query_record` 表中查询用户最近10条查询记录
2. **频率统计**：统计各类别的查询频率，过滤无效类别
3. **排序筛选**：按频率降序排列，取前3个高频类别
4. **内容生成**：根据类别生成对应的推荐内容（标题、描述、跳转目标）
5. **降级策略**：无查询记录时返回默认热门推荐

推荐配置

每个查询类别都有专属的推荐配置：

类别	图标	颜色	标题	描述	跳转目标
FACILITY_CANTEEN	Food	橙色	您可能感兴趣的食堂	探索更多校园美食	facility（食堂tab）
FACILITY_CAFE	Coffee	灰色	咖啡厅推荐	休闲放松的好去处	facility（咖啡厅tab）
FACILITY_LIBRARY	Reading	蓝色	图书馆推荐	安静的阅读和学习空间	facility（阅览室tab）
FACILITY_STUDY_ROOM	Notebook	绿色	自习室推荐	舒适的学习环境	facility（自习室tab）
FACILITY_LAB	Monitor	紫色	实验室推荐	探索科技创新空间	facility（实验室tab）
BUILDING	OfficeBuilding	绿色	校园建筑探索	了解复旦各校区建筑	campus
CAMPUS	MapLocation	红色	校区信息查询	探索复旦多个校区的风采	campus
COURSE	Collection	紫色	课程推荐	发现优质课程资源	course
TEACHER	User	青色	教师信息查询	了解授课教师信息	course
EVENT	Calendar	橙色	校园活动	精彩活动不错过	event

精准跳转

- 点击推荐项后，通过 `sessionStorage` 传递导航参数 (`targetPage`、`filterType`、`searchKeyword`)
- 目标页面接收参数后自动切换到对应的tab或执行搜索
- 例如：点击“食堂推荐”直接跳转到FacilityView的“食堂”tab